`jacobsrays`: ordered voxel intersections for tensor tomography and transport

Project documentation — user and methods guide

Draft, 28 October 2026

Abstract

Ray–voxel intersections provide a reusable geometric foundation for scalar and tensor tomography and for transport equations along rays. This guide describes a small C++11 implementation with Python and MATLAB interfaces, including integration with the Core Imaging Library (CIL). It specifies the coordinate and tensor conventions, explains how the same intersection weights produce matched forward and adjoint operators, and shows why ordered intervals are needed for non-commuting matrix transport. We document the connection to David Szotten's earlier code, independent validation against analytic calculations and ASTRA, and measured CPU performance, including an optional OpenMP atomic adjoint and experimental cached oneMKL products. Worked examples distinguish discrete consistency from continuum accuracy. This is practical software documentation, not a claim of a new ray-tracing algorithm or a general solution to tensor or non-Abelian inverse problems.

Repository: `https://github.com/billlion/jacobsrays`. The implementation is Apache-2.0 licensed. This draft describes main revision `854c6aa` (package version 0.1.0), including the optional atomic adjoint merged in PR 7. The atomic-adjoint measurements were made on revision `f6464f3` before that merge. Section 7.4 adds later CSF3 measurements of a benchmark-only oneMKL adapter; their raw records and provenance limitations are documented in `benchmarks/csf3/README.md`.

Reading this guide

For a first calculation, read the coordinate contract in Section 3 and the examples in Section 5. The Python listings are complete executable files; the MATLAB and CIL sections point to full examples in the repository.

For a new tensor or transport model, start with the operator conventions in Section 4. In particular, tensor packing, transverse trace and multiplication order are part of the numerical model.

For assessing results or choosing an implementation strategy, consult the validation evidence in Section 6 and the measured time and memory tradeoffs in Section 7. The limitations accompany each experiment so that reproducible consistency checks are not mistaken for reconstruction accuracy claims.

Appendix A starts a verification study with a precise specification and written exact-arithmetic correctness proof of the traversal. It introduces the proof terminology and a worked example; it is not yet a machine-checked or floating-point verification.

Contents

1 Why a small ray–voxel library? 3
- 1.1 Relation to ASTRA, TIGRE and CIL . . . 3

2 History and provenance 4

3 Geometry, storage and the traversal kernel 4
- 3.1 Coordinate contract . . . 4
- 3.2 Streaming the intersections . . . 5
- 3.3 Retain paths or recompute them? . . . 6

4 Operators, adjoints and tensor conventions 6
- 4.1 Scalar projection and a matched backprojector . . . 6
- 4.2 LRT and orthonormal packing . . . 7
- 4.3 Full rank-two TRT, trace tests and TTRT . . . 7
- 4.4 Ordered transport and non-Abelian models . . . 8

5 Interfaces and worked use 9
- 5.1 Python: paths, sparse matrices and matrix-free projection . . . 9
- 5.2 Python: a tensor with a known transverse trace . . . 10
- 5.3 MATLAB and compatibility with older scripts . . . 10
- 5.4 CIL solvers . . . 11

6 Validation: what has been checked? 11
- 6.1 Geometry and adjoint tests . . . 11
- 6.2 Independent comparison with ASTRA . . . 12
- 6.3 Slice-by-slice TRT: discrete consistency and continuum error . . . 13

7 Speed, memory and parallel accumulation 14
- 7.1 Recorded scalar benchmark . . . 14
- 7.2 Why the parallel adjoint is harder . . . 14
- 7.3 Optional OpenMP atomic scalar adjoint . . . 15
- 7.4 Experimental oneMKL acceleration of cached operators on CSF3 . . . 16

8 Reproduction and scope for further work 17

A Exact-arithmetic specification and correctness 19
- A.1 Inputs and the meaning of an output record . . . 19
- A.2 The exact algorithm corresponding to `walk()` . . . 21
- A.3 Correctness proof . . . 22
- A.4 A small example with a simultaneous crossing . . . 23
- A.5 What remains to be formalised . . . 23

References 24

1 Why a small ray–voxel library?

Many imaging models share a geometric question: which cells of a Cartesian grid does a straight ray cross, and for how long does it remain in each? For a piecewise-constant scalar field, these lengths are the coefficients of a discrete X-ray transform. For a tensor field, the same coefficients can be combined with contractions or projections depending on the ray direction. For a transport equation, the entry and exit points define successive integration intervals. Keeping the geometry separate makes it possible to change the measurement model without rewriting traversal.

This separation is useful in experimental tensor imaging: one may need a new tensor contraction, a polarisation readout, an unusual set of rays, or the derivative of a nonlinear forward model. A compact, inspectable CPU kernel also provides a reference against which more specialised implementations can be checked. It does not replace a complete imaging toolbox, detector model or reconstruction method.

The main supported uses are the scalar X-ray transform, the longitudinal ray transform (LRT) of symmetric tensors, and the full transverse ray transform (TRT) of symmetric rank-two tensors. The truncated transverse ray transform (TTRT), used in linearised photoelastic tomography, can be built from the same geometry and transverse projection. The distinction between full and truncated data matters: the latter removes the transverse trace. Section 4 gives the precise conventions and implementation status. ORT is a proposed further application; this draft does not assign it a definition or claim an implementation. A public specification and validation example are needed before adding it to the API.

Non-Abelian ray transforms require additional information that scalar matrix storage need not preserve: *travel order*. Rotations or other matrix propagators generally do not commute, so changing the order of encountered voxels changes the result. Polarimetric neutron imaging is a concrete application [6, 14]. The package returns ordered intervals directly rather than requiring users to infer order from a sparse matrix.

1.1 Relation to ASTRA, TIGRE and CIL

ASTRA [19] and TIGRE [2] provide mature tomography infrastructure and GPU projectors. They are natural choices for many large scalar imaging problems. `jacobsrays` currently supplies a CPU kernel, explicit segment geometry, and a small set of scalar and tensor operators. Its useful distinguishing feature here is access to ordered cell intervals and a common traversal for custom forward models.

Comparisons must specify a projector, not just a toolbox name. ASTRA's 2D CPU `line` projector uses intersection lengths; its `linear` projector uses a Joseph-type interpolation model [1]. TIGRE also exposes different forward and backward models. For example, the documented CIL TIGRE adapter distinguishes Siddon and interpolated forward projection, and describes its matched adjoint weights as approximately matching the interpolated method [4]. This does not establish an exact transpose for every possible pairing. Independent numerical checks should follow the chosen configuration.

CIL [10] provides optimisation algorithms and data containers into which a new operator can be inserted. Our adapter therefore implements CIL's operator interface directly, without requiring ASTRA or TIGRE as a backend. No comparative GPU speed result is claimed in this guide.

2 History and provenance

The underlying traversal belongs to the Siddon/Jacobs family. Siddon [16] describes the grid through its planes rather than testing every voxel separately. Jacobs et al. [9] describe an efficient calculation of the radiological path. The present implementation merges the ordered sequences of plane crossings; it does not claim a new geometric algorithm.

David Szotten developed a MATLAB-wrapped implementation at Manchester; the source preserved in `LRTinMatlab` carries 2005–2008 attribution. His thesis concerns limited-data X-ray and polarised-light tomography, including the TTRT [17]. The university catalogue records the award on 3 December 2010, while the later papers cite the thesis as 2011; the bibliography follows those citations and records this distinction.

The historical connection is documented in several public sources:

- The 2006 MIMS preprint by Szotten, Lionheart and Tomlinson describes using Jacobs intersection lengths to construct scalar, transverse and truncated transverse matrices. It also describes accumulating the transpose ray by ray when storing the full matrix is impractical [18]. It is cited here as a preprint, not as a subsequently published journal article.
- Desai and Lionheart's three-axis TRT reconstruction paper provides the published tensor phantom used in our slice example. Its numerical method section explicitly follows the ray integration procedure in Section 5.1.4 of Szotten's thesis [5]. The MATLAB research-code repository preserves the older ray implementation [11].
- Section 7 of Desai et al.'s polarimetric-neutron paper explicitly states that its forward solver used the Jacobs method as implemented in Szotten's thesis, modified to return voxel encounter order. Voxelwise constant magnetic fields then give successive rotation matrices along the ray [6]. The experimental context is described by Sales et al. [14].

This history explains the present emphasis on both lengths and ordering. The new C++ implementation was written afresh after an audit of the older interface and behaviour. The inspected legacy tree did not contain a licence granting redistribution of its code; no legacy source is bundled or relicensed. The repository's `NOTICE` records the intellectual provenance and development assistance.

The audit found practical reasons to make the conventions explicit. In the old implementation, non-square grids could receive an incorrect linear index, the final partial segment could be mishandled, and some reported weights were parameter intervals rather than physical lengths. A MATLAB sparse-sizing idiom could overwrite a genuine final coefficient. The legacy help text also disagreed with the implementation about the grid offset. These are recorded with small examples in `docs/legacy-audit.md`; they are not reasons to treat all historical numerical results as invalid. The compatibility wrapper preserves the old calling convention while using the corrected, documented geometry.

3 Geometry, storage and the traversal kernel

3.1 Coordinate contract

Let the grid have shape (n_x, n_y, n_z), positive spacing $h = (h_x, h_y, h_z)$ and lower corner o. Voxel (i, j, k) is

$$[o_x + ih_x, o_x + (i+1)h_x) \times [o_y + jh_y, o_y + (j+1)h_y) \times [o_z + kh_z, o_z + (k+1)h_z).$$

The zero-based linear index is $i + n_x(j + n_y k)$. Thus x is the fastest varying coordinate. Two-dimensional grids use the same rules with the z coordinate omitted at the public interface.

Inputs are finite segments

$$x_r(t) = a_r + t(b_r - a_r), \qquad 0 \le t \le 1. \tag{1}$$

Endpoints need not lie outside the grid. To approximate an infinite line through the object, choose endpoints enclosing its entire intersection with the grid. If an emitted interval is $[t_0, t_1]$, its physical length is

$$\ell = (t_1 - t_0)\|b_r - a_r\|_2. \tag{2}$$

The parameters themselves are dimensionless, not arclengths.

Convention	Python	MATLAB
Geometry arguments	$(n_x, n_y[, n_z])$, xyz endpoints	Same
Image array shape	(n_z, n_y, n_x) or (n_y, n_x)	$[n_x, n_y, n_z]$ or $[n_x, n_y]$
Linear storage	C order; x fastest	Column major; x fastest
Indices and offsets	Zero based	One based
Scalar range	One number per input ray	One number per input ray
Empty/missing rays	Repeated offsets; zero matrix rows	Same meaning

Table 1: Coordinate and storage conventions. Array shape reversal is deliberate; it does not reverse the physical coordinate axes.

A parallel ray on an internal cell plane belongs to the higher-index cell. The lower exterior face is included and the upper exterior face excluded. Point tangencies, misses and zero-length segments emit no positive-length intervals. These boundary choices have measure zero in continuum theory but are essential for reproducible discrete tests.

3.2 Streaming the intersections

The computational core is `include/jacobsrays/traverse.hpp`. It is a header because the central `walk<Emit>` routine is templated on its output action. Keeping its definition visible lets the compiler specialise and inline different actions without a virtual call at every intersection. Header implementation is a C++ design choice, not an assumption that headers execute faster.

Conceptually the traversal performs the following operations:

1. Clip the segment to the grid box by intersecting its parameter interval with the coordinate slabs. Return if there is no positive-length overlap.
2. Find the cell at entry and the next grid-plane crossing for each axis. An axis parallel to the ray contributes no finite next crossing.
3. Take the earliest next crossing, or the clipped segment exit if sooner. Emit the current cell index, physical length, and entry/exit parameters.
4. Advance every axis tied at that crossing, update its next plane, and continue until exit.

This merges two or three monotone crossing sequences without sorting a list of all crossings. Work is $O(K_r)$ for K_r crossed cells, with $O(1)$ traversal workspace per ray. Next crossing parameters are recomputed from integer plane positions to limit cumulative drift. Exact ties advance all tied axes. Numerically near-tied crossings can produce very short positive intervals; the code does not silently merge them using an arbitrary absolute tolerance. Reversal is consequently checked numerically, not promised to be bitwise identical.

Finite inputs, representable grid extents and integer index ranges are checked. There is no absolute small-direction cutoff such as 10^{-8}; such a cutoff would depend on the user's physical units. Standard compiler optimisation is appropriate. Unrestricted fast-math transformations are not assumed because the algorithm uses finite-value and infinity semantics.

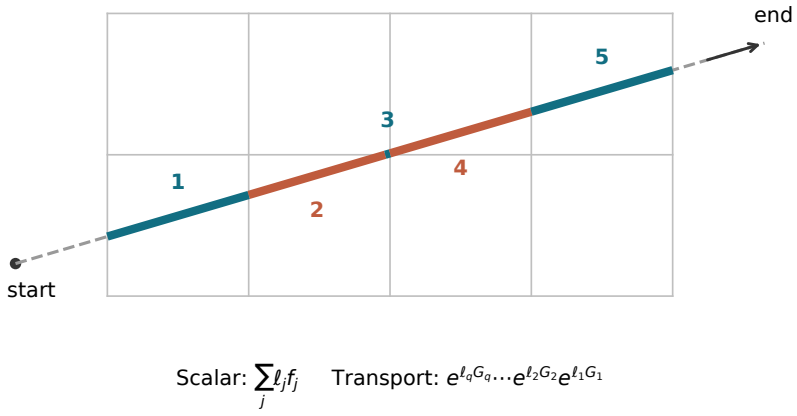


Figure 1: A finite segment clipped to a grid, with intervals numbered in travel order. Scalar projection sums length-weighted values. Matrix transport applies the first propagator first, placing later factors to its left for the convention $U' = GU$. Cell numbers in the sketch are encounter labels, not linear storage indices.

3.3 Retain paths or recompute them?

`trace` returns ragged ordered arrays: `offsets`, `indices`, `lengths`, `t_enter` and `t_exit`. Entries for ray r are selected by successive offsets, in start-to-end order. Native batch tracing counts intersections and then fills exactly sized arrays in a second pass. The Python wrapper releases the GIL during native computation.

For K total intersections and R rays, the returned arrays occupy $32K + 8(R + 1)$ bytes with 64-bit indices and double precision lengths and parameters. This excludes object headers and input geometry. Batching rays limits this storage. Conversion to a scalar sparse matrix discards the entry/exit parameters, and subsequent sparse operations may reorder columns. Use the original paths for transport ODEs.

The scalar `RayTransform` instead retraces rays on each application, avoiding persistent intersection storage. Python tensor operators cache one scalar CSR matrix, and combine it with small direction-dependent operations. MATLAB exposes sparse matrices. These choices trade setup cost and memory against repeated-iteration speed; no single choice wins for every acquisition.

4 Operators, adjoints and tensor conventions

4.1 Scalar projection and a matched backprojector

For piecewise-constant voxel values x_i, define

$$A_{ri} = \text{length}(\text{segment } r \cap \text{voxel } i), \qquad (Ax)_r = \sum_i A_{ri} x_i, \qquad (A^{\mathsf{T}} y)_i = \sum_r A_{ri} y_r. \tag{3}$$

The native forward callback adds ℓx_i to one ray output; the backprojection callback adds ℓy_r to one voxel output. Both call the same traversal with the same endpoints. Backprojection does not reverse the ray and does not substitute another interpolation or quadrature rule.

CIL's `JacobProjector.adjoint` method is defined in `python/jacobsrays/cil.py`. It calls the main Python module's `RayTransform.adjoint` method, defined in `python/jacobsrays/__init__.py`. The native backprojector in `src/python.cpp` then invokes `walk` in `traverse.hpp`. Tensor adjoints follow their cached scalar matrix transpose instead of this matrix-free native path.

This is a *matched discrete transpose*, up to floating-point arithmetic. It is the adjoint for ordinary Euclidean array dot products. For weighted inner products with positive definite matrices W_X and W_Y, the corresponding adjoint is $W_X^{-1}A^{\mathsf{T}}W_Y$. Voxel volumes, detector areas and angular quadrature weights are not inserted automatically.

An unmatched pair uses a backward operator $B \neq A^{\mathsf{T}}$. Such pairs can be useful computationally, but replacing A^{T} by B changes the algebra of an iterative method: BA need not be symmetric positive semidefinite and $B(Ax - y)$ need not be the least-squares gradient. The usual CGLS reasoning therefore no longer applies. Perturbation and convergence analysis, and methods designed for unmatched pairs, are discussed in Elfving and Hansen [7], Hansen et al. [8]. Neither hardware texture interpolation nor GPU execution alone establishes that a particular pair is unmatched; the actual weights and their transpose must be examined.

A matched pair still does not guarantee an accurate or unique reconstruction. Nullspaces, acquisition coverage, noise, discretisation and regularisation remain properties of the inverse problem.

4.2 LRT and orthonormal packing

For a symmetric rank-m tensor field f and an oriented unit ray direction u, the LRT is

$$(I_m f)(r) = \int_r f(x)[u, \ldots, u]\,\mathrm{d}s. \tag{4}$$

Let $\alpha = (\alpha_1, \ldots, \alpha_d)$ with $|\alpha| = m$, and $c_\alpha = m!/\alpha!$. Store the tensor component as $x_\alpha = \sqrt{c_\alpha} f_\alpha$, and define $w_{r\alpha} = \sqrt{c_\alpha} u_r^\alpha$. Then

$$(Lx)_r = \sum_\alpha w_{r\alpha}(Ax_\alpha)_r, \qquad (L^{\mathsf{T}}y)_\alpha = A^{\mathsf{T}}(w_{\cdot\alpha} \odot y). \tag{5}$$

This orthonormal packing makes an ordinary component dot product equal to the tensor Frobenius product, including the multiplicities of off-diagonal components. In 3D at rank two it is

$$\mathrm{pack}(F) = (F_{11}, \sqrt{2}F_{12}, \sqrt{2}F_{13}, F_{22}, \sqrt{2}F_{23}, F_{33}). \tag{6}$$

In 2D it is $(F_{11}, \sqrt{2}F_{12}, F_{22})$. `TensorRayTransform` implements these formulas in 2D and 3D for nonnegative integer rank; its `exponents` attribute records the component order. Reversing the endpoints changes the LRT by $(-1)^m$. The familiar potential-field nullspace of the continuum LRT is not removed by this discretisation [15].

4.3 Full rank-two TRT, trace tests and TTRT

Let $P_u = I - uu^{\mathsf{T}}$. The implemented full TRT is

$$(Jf)(r) = \int_r P_{u_r} f(x) P_{u_r}\,\mathrm{d}s. \tag{7}$$

`TransverseRayTransform` uses the packing in (6) in both domain and range. Its 3D input has shape $(6, n_z, n_y, n_x)$ and output has shape $(R, 6)$. The six output entries encode only three independent transverse components; retaining a Cartesian representation avoids an arbitrary

detector-frame choice. If E contains two orthonormal transverse detector axes, the detector matrix is $E^{\mathsf{T}}(Jf)E$.

For each ray the map $F \mapsto P_u F P_u$ is self-adjoint in the Frobenius product. Thus the discrete adjoint applies this map to the input tensor data and backprojects each resulting component with A^{T}. It is defined even for data that are not already transverse. Small per-ray projectors and one scalar CSR matrix are retained; a dense block tensor matrix is not assembled in Python.

Two useful identities follow directly:

$$\operatorname{tr}(Jf) = X(\operatorname{tr} f) - I_2 f, \qquad J(\phi I) = (X\phi)P_u. \tag{8}$$

Here X denotes the scalar line integral along the same ray. Consequently $\operatorname{tr} J(\phi I) = (d-1)X\phi$. These test the retained transverse trace as well as the component packing. The full TRT is relevant to published tensor imaging models including diffraction tomography of strain [13] and the reconstruction analysis of Desai and Lionheart [5].

For rank two in 3D, define the transverse trace-free map

$$Q_u(F) = P_u F P_u - \frac{\operatorname{tr}(P_u F P_u)}{2} P_u. \tag{9}$$

The TTRT integrates this quantity [12, 18]. The divisor is the transverse dimension, two, not the ambient dimension three. Isotropic fields are invisible: $Q_u(\phi I) = 0$. "Truncated" in this usage concerns the transverse trace; it does not mean a spatially truncated field of view. The current API implements the full TRT, not a separate TTRT class. A future adapter can apply Q_u to the full TRT output and use the same self-adjoint projection in its backward map. This guide does not introduce unpublished Fourier inversion formulas or equate three-axis data with complete directional data.

4.4 Ordered transport and non-Abelian models

Suppose a matrix propagator satisfies

$$\frac{\mathrm{d}U}{\mathrm{d}s} = G(x(s), u)U(s), \qquad U(s_{\text{entry}}) = I, \tag{10}$$

where s is physical arclength. With constant generator G_j in the jth encountered voxel and interval length ℓ_j,

$$U_{\text{exit}} = e^{\ell_q G_q} \cdots e^{\ell_2 G_2} e^{\ell_1 G_1}. \tag{11}$$

The latest factor is leftmost for this ODE convention. In general this is not $\exp(\sum_j \ell_j G_j)$. The geometric traversal is exact for the specified cells; matrix-exponential evaluation has numerical error, and approximating a smooth field by voxel constants has model error.

The example `examples/nonabelian_transport.py` implements this product with SciPy matrix exponentials. It is a generic forward-model building block, not a complete neutron or optical instrument model. For variable within-voxel coefficients, recover the physical entry/exit points from (1) and use an ODE solver on each interval. A connection uses a direction-dependent generator: reversal changes both traversal order and the contracted generator sign. Reversing endpoints alone for an orientation-independent $G(x)$ does not generally produce inverse transport.

The forward map in (11) is nonlinear in the generators. Its linearisation contains products before and after each perturbation; the scalar backprojector is not automatically its Jacobian adjoint. That derivative and its tests belong in any future nonlinear reconstruction implementation. This separation follows the physical motivation of the polarimetric-neutron work without claiming to reproduce its full inversion.

5 Interfaces and worked use

5.1 Python: paths, sparse matrices and matrix-free projection

From a checkout with a C++ compiler and Python 3.10 or newer:

```
python -m pip install '.[scipy]'
```

NumPy is the mandatory runtime dependency. SciPy is needed for sparse matrices, tensor operators, iterative solves and the matrix-exponential example. The following is a complete small scalar example:

```
"""Complete scalar example printed in the user guide."""
import numpy as np
from jacobsrays import Grid, RayTransform, trace

grid = Grid((3, 2), spacing=(1, 1), origin=(0, 0))
starts, ends = [[-1, .5]], [[4, .5]]
paths = trace(grid, starts, ends)
k = paths.ray(0)
print(paths.indices[k], paths.lengths[k])

A = RayTransform(grid, starts, ends)
x = np.ones(grid.image_shape)  # (ny, nx)
y = A.direct(x)
xadj = A.adjoint(y)
matrix = paths.to_csr()
np.testing.assert_allclose(y, matrix @ x.ravel())
np.testing.assert_allclose(xadj.ravel(), matrix.T @ y)
print(y)
```

It prints indices $0, 1, 2$, three unit lengths, and the scalar projection 3. The scalar API uses real double-precision arrays. Complex generators can be used in the Python transport example, but the scalar and tensor projector APIs do not currently accept complex image data.

5.2 Python: a tensor with a known transverse trace

The following uses $f = \phi I$ on a small 3D grid. It checks the trace identity without constructing a full tensor system matrix:

```
"""Complete full-TRT trace example printed in the user guide."""
import numpy as np
from jacobsrays import Grid, RayTransform, TransverseRayTransform

grid = Grid((4, 3, 2))
starts = [[-1, .5, .5], [.5, -1, .5]]
ends = [[5, .5, .5], [.5, 4, .5]]
phi = np.ones(grid.image_shape)
F = np.zeros((6, *grid.image_shape))
F[[0, 3, 5]] = phi  # physical tensor phi*I

J = TransverseRayTransform(grid, starts, ends)
Y = J.direct(F)
X = RayTransform(grid, starts, ends).direct(phi)
np.testing.assert_allclose(Y[:, [0, 3, 5]].sum(axis=1), 2*X)
print("TRT trace:", Y[:, [0, 3, 5]].sum(axis=1))
```

For more varied fields, `examples/tensor_trt.py` uses a spatial mask and an anisotropic tensor, checks transversality and the adjoint identity, and can reconstruct with CIL. `examples/tensor_lrt.py` demonstrates the longitudinal operator and its rank-dependent packing.

5.3 MATLAB and compatibility with older scripts

```
addpath('matlab');
build_jacobsrays;
test_jacobsrays;
test_trt;
p = jacobsrays([3 2], [-1 .5], [4 .5]);
A = jacobsrays_matrix(p);
x = ones(3,2);
y = A*x(:);
xadj = reshape(A'*y, [3 2]);
k = p.offsets(1):p.offsets(2)-1;
disp([p.indices(k), p.lengths(k)]);
```

The MEX layer uses the same C++ header. Inputs are full real double arrays; the path indices and offsets returned to MATLAB are one based. `jacobsrays_matrix` preserves empty rows and the requested full shape, including a zero-ray matrix. MATLAB's sparse transpose supplies the matched scalar adjoint.

For rank-two TRT, `jacobsrays_trt_matrix` explicitly builds the block sparse matrix T. With voxel components stored as an $N \times c$ matrix F, and ray components as an $R \times c$ matrix Y, use

```
[T, paths] = jacobsrays_trt_matrix(shape, starts, ends, spacing, origin);
Y = reshape(T*F(:), size(starts,1), size(F,2));
Fadj = reshape(T'*Y(:), prod(shape), size(F,2));
```

Here $c = 6$ in 3D and $c = 3$ in 2D, with the orthonormal components already packed. MATLAB vectorisation is component-major in both spaces; Python's TRT output is an $R \times c$ C-order array.

Do not transfer flattened tensor arrays between languages without respecting this distinction. The explicit 3D tensor matrix can have up to 36 entries per scalar intersection, so it is intended for modest problems. A complete runnable MATLAB example is `matlab/example_trt.m`.

The compatibility function `mod_jacobs_ray3d_multiple` preserves the old signature, including ordered `'internal'` triples. Its offset means the lower grid corner, matching the legacy implementation rather than the contradictory old help text. Put the new MATLAB directory first on the search path and check geometry conventions when porting scripts.

5.4 CIL solvers

CIL is installed separately using its own instructions [3]. The adapter provides `direct`, `adjoint` and optional `out` arguments. Scalar `JacobProjector` maps a single-channel `ImageGeometry` to a `VectorGeometry` with one entry per supplied ray. It currently requires the default spatial dimension labels and no singleton spatial dimensions. It does not infer rays from a CIL `AcquisitionGeometry`.

With an image geometry `ig`, explicit endpoints and compatible data, the central CGLS calls are:

```
from cil.optimisation.algorithms import CGLS
from jacobsrays.cil import JacobProjector
operator = JacobProjector(ig, starts, ends)
solver = CGLS(initial=ig.allocate(0, dtype=np.float64),
              operator=operator, data=data)
solver.run(30, verbose=0)
image = solver.solution
```

`examples/cil_cgls.py` supplies the complete geometry and data setup. Its recorded noiseless demonstration reached a relative data residual of about 1.68×10^{-4} after 30 iterations. This is an example outcome, not a general convergence guarantee or a recommended stopping rule for noisy measurements.

`TensorJacobProjector` has a block-image domain and a scalar-vector range. The TRT adapter, `TransverseJacobProjector`, uses component blocks in both domain and range. Run

```
python examples/cil_cgls.py
python examples/tensor_lrt.py
python examples/tensor_trt.py --cil
python examples/nonabelian_transport.py
```

The CIL adapter is imported only when requested. Tensor operators use cached scalar matrices; they do not route their adjoints through the native atomic scalar implementation.

6 Validation: what has been checked?

Validation is layered because a dot-product test alone can pass for a consistently wrong forward model. Conversely, comparing two legitimate but different discretisations need not give roundoff-level agreement.

6.1 Geometry and adjoint tests

The Python reference oracle enumerates plane intersections and sorts them; it does not reuse the streaming walk. Cases include anisotropic spacing, offset origins, non-cubic grids, reversed

Layer	Evidence	Main limitation
Traversal	4,000 random 2D/3D segments against an independent plane-enumeration oracle; hand cases and edge cases	Finite sampling of floating-point inputs
Operator algebra	Sparse versus matrix-free results; transpose dot products; LRT ranks 0–3 and TRT component checks	Shared geometry can hide common errors
Tensor physics	Independent per-voxel clipping and tensor formula; transversality, isotropic and trace identities	Tests the stated model, not every instrument
External scalar code	ASTRA CPU line matrices, projections and adjoints; CIL/ASTRA geometry integration	2D, square pixels, single precision reference
Continuum example	Analytic Gaussian line integrals and separate numerical quadrature; slice reconstruction	Specific phantom and fixed acquisition
Language bindings	C++ tests, Python tests, MATLAB MEX tests in CI	No claim of all compilers/platforms

Table 2: Different checks answer different questions. Exact geometry, adjointness, physical modelling and inverse-problem accuracy are distinct.

and partial rays, simultaneous crossings, parallel rays on boundaries, misses, tangencies, empty batches, zero-length segments and invalid inputs. C++ tests exercise the standalone header. Address and undefined-behaviour sanitizer checks have also been used during development; this is not a claim of exhaustive verification.

For real arrays the adjoint identity is $\langle Ax, y\rangle = \langle x, A^{\mathsf{T}}y\rangle$. A useful diagnostic is

$$\varepsilon_{\mathrm{adj}} = \frac{|\langle Ax, y\rangle - \langle x, A^{\mathsf{T}}y\rangle|}{\|Ax\|\|y\| + \|x\|\|A^{\mathsf{T}}y\|}, \tag{12}$$

with zero treated separately when the denominator vanishes. This avoids dividing only by a dot product that may nearly cancel. Tests also compare against explicit sparse products. TRT tests use arbitrary tensor data in the adjoint, so an incorrect assumption that all input data are already transverse is detectable.

The atomic extension adds high-contention tests: 16,384 identical rays accumulate into the same four voxels using exactly representable unit lengths and small integer data. Random anisotropic 2D/3D cases compare serial, atomic and sparse results; CIL tests exercise `out` and CGLS. At the tested PR revision the OpenMP suite had 29 passing tests; the serial build had 25 passes and four OpenMP-only skips. Counts are a dated snapshot, not a coverage metric. MATLAB tests run separately on GitHub Actions; no local MATLAB installation was used to build this document.

6.2 Independent comparison with ASTRA

The external tests use ASTRA 2.5.0 CPU `line` and `line_fanflat` projectors. The direct comparison includes six geometries, each with 29 angles and 41 detector samples, for 7,134 rays in total. Images have 11×8 pixels, with square-pixel widths 1, 0.375 and 2.5 and shifted windows. Full system matrices are compared before forward and transpose products. ASTRA's row ordering and this package's physical y convention are reconciled explicitly; an asymmetric image helps expose accidental flips. Separate CIL/ASTRA tests cover parallel and fan geometries, detector orientation and angular offsets.

ASTRA's reference values are single precision; our traversal uses doubles. The coefficient tolerance is 3×10^{-4} times pixel width, with relative L^2 tolerance 2×10^{-5} for the corresponding projection checks. The recorded worst coefficient discrepancy was approximately 1.79×10^{-4} pixel widths; forward and adjoint discrepancies were of order 10^{-6} to 10^{-5}. These tolerances apply to these tests, not a bound for all geometries.

This comparison does not validate 3D external projectors, tensor contractions, travel order, or GPU interpolation kernels. Non-square pixels are excluded from the ASTRA CPU line reference comparison; anisotropic geometry is instead checked by the independent internal oracle. TIGRE is discussed as a relevant toolbox but has not yet been used as a validation or timing reference in this repository.

6.3 Slice-by-slice TRT: discrete consistency and continuum error

If a ray lies in an x_3-constant plane, then $u_3 = 0$ and $P_u e_3 = e_3$. It follows for any symmetric field that

$$(Jf)_{33}(r) = \int_r f_{33}(x)\,\mathrm{d}s. \tag{13}$$

Thus a scalar 2D reconstruction can recover this component on the plane from these TRT data, subject to the usual sampling and inversion issues. Choosing voxel-centre planes avoids inter-slice interpolation.

`examples/trt_slices.py` uses only the published f_{33} component from Table 1 of Desai and Lionheart [5]. On $[-1, 1)^3$ it is

$$\begin{aligned} f_{33}(x) = {} & e^{-50\|x-(.5,-.5,-.5)\|^2} - e^{-50\|x-(.5,-.5,.5)\|^2} \\ & + e^{-50\|x-(.5,.5,.5)\|^2}, \end{aligned} \tag{14}$$

and is zero outside; the other tensor components are set to zero here. This example does not reproduce the paper's full tensor phantom or three-axis inversion algorithm.

Two layers centred at $x_3 = \pm 0.5$ contain $n \times n$ centre samples. The default acquisition uses 90 angles over a half-turn with angular offset 0.013 radians, and 129 detector offsets from -1.45 to 1.45. Three data sets are compared: the 33 output of the voxel TRT, a separate 2D scalar projection of the samples, and analytic integrals of the continuous Gaussian sum clipped to the cube and finite segments. The last use error functions and are independently checked against adaptive quadrature.

Each data set is inverted with the same scalar matrix and unregularised LSMR, tolerances `atol=btol=1e-10` and at most 300 iterations. The reference is the sampled continuous field, not voxel averages.

Slice grid	Voxel/analytic data error relative L^2	Image error from analytic data $x_3 = -0.5$	$x_3 = +0.5$
32×32	4.13%	2.79%	2.77%
64×64	1.48%	4.62%	5.32%

Table 3: Recorded slice example (NumPy 2.3.5, SciPy 1.17.0). Smaller forward modelling error does not imply a smaller unregularised reconstruction error when acquisition sampling is held fixed.

TRT and scalar voxel data agreed exactly in that run, as did their reconstructions. Matched voxel-data image errors were approximately 2×10^{-9} and 2×10^{-8} at the two resolutions. This is a discrete consistency check, often called an inverse crime, rather than independent evidence of

continuum reconstruction accuracy. All default solves stopped on tolerance before the iteration limit.

The analytic-data image errors increase under this refinement even though the data discrepancy decreases. Fixed acquisition sampling, conditioning and model mismatch can all affect the inverse result; this experiment does not isolate them. Regularisation and controlled refinement of both image and acquisition are needed for a convergence study. The small CI regression therefore tests identities and solver consistency rather than imposing a false monotonic error requirement.

7 Speed, memory and parallel accumulation

7.1 Recorded scalar benchmark

The recorded 26 October 2026 benchmark used Linux, an AMD EPYC 9V74 CPU, GCC 13.3.0, Python 3.12.14 and NumPy 2.3.5. There were 20,000 segments on a 64^3 grid and 2,107,166 intersections. Times below are the best of five warm repetitions and include per-call validation and output allocation. CSR timings exclude construction of the matrix. The ordered

Operation	Serial build (ms)	OpenMP, 4 threads (ms)
Ordered paths, two passes	46.42	9.90
Matrix-free forward	23.33	5.48
Matrix-free adjoint, serial accumulation	24.45	24.48
Cached CSR forward	1.63	1.62
Cached CSR transpose	1.94	2.08

Table 4: Scalar benchmark before the atomic extension. The CSR operations use SciPy and do not acquire a four-thread traversal merely because the native extension was built with OpenMP.

arrays occupied 67,589,320 bytes; the SciPy CSR matrix occupied 25,365,996 bytes in that run, including its 32-bit index arrays. CSR is smaller because it omits the interval endpoints and uses narrower indices for this problem. It also amortises geometry over repeated iterations. The matrix-free method is useful when storage or changing ray geometry makes caching undesirable.

A separate one-pass native comparison used 200,000 rays with 21,068,823 intersections. The legacy C traversal took 137.70 ms and the new C++ traversal 164.20 ms, approximately 19% longer. Intersection counts and the recorded length checksum agreed for that geometry. This test excludes wrappers and two-pass path allocation. It shows that the rewrite is not intrinsically faster in serial: validation, robust indexing and floating-point choices have costs. It is not evidence of a general C versus C++ language advantage. Whole-workflow memory, caching and parallelism matter more than a language label.

These results are stored in `benchmarks/results_2026-10-26.json`. They are observations on one shared environment, not universal speed claims. No speed comparison with ASTRA, TIGRE or a GPU has been measured.

7.2 Why the parallel adjoint is harder

Forward projection parallelises over rays: each worker reads image values and writes its own ray output. The transpose scatters contributions into the image. Multiple rays can update the same voxel simultaneously, and an unsynchronised read–modify–write can lose updates.

Three common approaches are atomic additions to one image, private images per thread followed by reduction, and a decomposition that assigns output regions to workers. Atomics

save replicated image memory but contend for popular voxels and cache lines. Private images cost roughly one image per thread plus reduction work. Spatial decompositions require additional indexing or ray work. This is a shared-memory OpenMP issue; MPI is not used by the current library.

7.3 Optional OpenMP atomic scalar adjoint

At revision `f6464f3`, the opt-in native scalar backprojector uses an OpenMP atomic addition for each intersection. It writes one shared image; it does not allocate a complete image per thread. The default remains serial accumulation with fixed ray order. Atomic mode preserves the same coefficients in (3), but summation order can change with scheduling, so the output need not be bitwise reproducible.

Use a fresh checkout or remove the generated build directory when changing compiler flags, then build and select the mode:

```
JACOBSRAYS_OPENMP=1 python -m pip install --no-cache-dir '.[scipy]'
export OMP_NUM_THREADS=4
export OMP_DYNAMIC=FALSE
```

```
A = RayTransform(grid, starts, ends, adjoint_mode="atomic")
xadj = A.adjoint(data)
```

The same keyword is accepted by scalar `JacobProjector`; the SciPy operator uses the selected adjoint too. An atomic request in a serial build raises an error. Tensor CSR operations and MATLAB sparse products are unaffected. A compiler and runtime with OpenMP support are required; the environment variables must be set before starting Python.

The 27 October experiment compared serial and atomic adjoints inside each fresh process, with 1, 2, 4 or 8 requested threads, fixed geometry/seed, one warm-up and five alternating measured repetitions. BLAS thread counts were fixed to one. The process had nine CPUs available by affinity. Both methods include image allocation and input checking. Median timings are used here, so they must not be compared as regressions against the earlier minimum timings.

Ray distribution	Threads	Serial (ms)	Atomic (ms)	Ratio
Distributed	1	30.60	22.64	1.35
Distributed	4	28.55	9.51	3.00
Distributed	8	27.35	6.07	4.51
Through centre	4	30.10	12.15	2.48
Through centre	8	31.24	9.62	3.25
Identical rays	4	9.51	5.04	1.89
Identical rays	8	9.69	7.62	1.27

Table 5: Selected paired medians for 20,000 rays on a 64^3 grid. Ratio is serial time divided by atomic time in the same run. A one-thread ratio above one can reflect compiled-loop and timing differences; it is not parallel scaling. Full measurements accompany the source.

The largest observed relative image difference from serial was 7.1×10^{-15} and the largest norm-scaled adjoint discrepancy (12) was 1.1×10^{-17}. These are measured values, not universal roundoff bounds. Geometry-dependent contention explains why thread count should be measured rather than simply maximised. The source experiment is `benchmarks/atomic_adjoint.py`; its recorded JSON is `benchmarks/atomic_results_2026-10-27.json`. Both were added in PR 7. Building the guide reads this JSON without rerunning the benchmark.

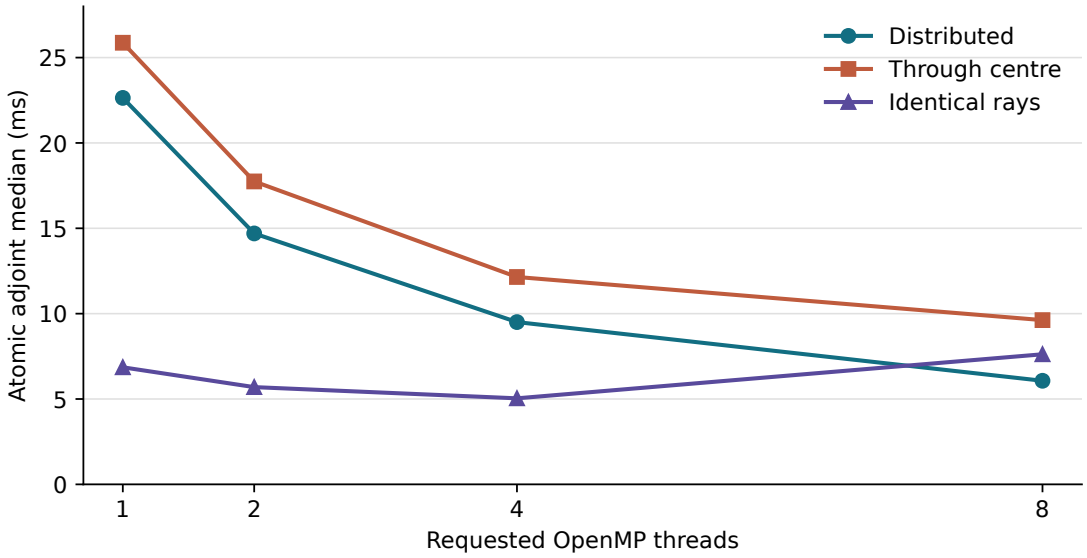


Figure 2: Atomic-adjoint median time versus requested threads, from the recorded experiment. Identical rays concentrate writes onto the same voxels: eight threads were slower than four for this case. These curves describe a single environment and benchmark, not confidence intervals.

7.4 Experimental oneMKL acceleration of cached operators on CSF3

An additional experiment on 27 October 2026 tested Intel oneMKL Sparse BLAS on the University of Manchester CSF3 cluster. The benchmark-only adapter in `benchmarks/mkl_probe.py` replaces products with a cached scalar CSR matrix by oneMKL products, including products with six tensor components. The LRT and TRT timings include the surrounding component packing, contractions and transverse projections. These measurements concern cached operators; they do not measure the matrix-free atomic backprojector or a complete iterative reconstruction. The adapter remains experimental and does not add a supported MKL backend to the package API.

Both repeated jobs requested 16 CPUs on one node and reported an AMD EPYC 9634 processor. They used Python 3.12.16, NumPy 2.3.5, SciPy 1.17.0 and oneMKL 2026.2.0, with the GNU threading layer, LP64 indices and passive OpenMP waiting. Each requested thread count ran in a fresh process; dynamic OpenMP/MKL thread adjustment was disabled. The problem comprised a 128^3 grid and 100,000 random segments entering through one x-face and leaving through the opposite face, with fixed random seed 20261027. The scalar matrix had 21,200,180 nonzeros. This geometry is a performance test, not a claim of adequate angular sampling for inversion.

Each product was evaluated before timing for a correctness check, followed by nine timed repetitions with rotating and reversing backend order. Table 6 reports the sum of the separate forward and adjoint median times. Geometry construction and backend preparation are excluded; output allocation and the operator's per-call work are included. The SciPy controls comprise the original CSR matrix, sorted CSR, and sorted CSR with a separately cached CSR transpose. SciPy sparse products in these controls are single-threaded.

For the eight-thread comparisons, the SciPy cached-transpose pair costs were respectively $(106.26, 588.70, 625.17)$ ms in run A and $(78.62, 579.04, 616.35)$ ms in run B, in scalar/LRT/TRT order. For the scalar case this combines the sorted-CSR forward time with the cached-transpose adjoint time. The corresponding gains from eight-thread MKL were 5.31 and 6.34 for scalar

	Run A			Run B		
MKL threads	Scalar	LRT	TRT	Scalar	LRT	TRT
1	100.47	370.40	418.15	64.37	358.20	391.21
2	32.64	224.82	244.77	39.81	206.22	261.47
4	25.14	156.87	158.88	19.48	115.26	161.24
8	20.01	97.06	134.44	12.39	77.22	117.38
16	11.87	91.12	121.14	15.37	81.96	117.79

Table 6: Cached-operator forward-plus-adjoint costs in milliseconds on CSF3. Runs A and B are separate submissions of the same job script, recorded in `mkl-23867575.json` and `mkl-23867786.json`. Each entry sums two separate medians; it is not a measured solver iteration. The two jobs had different permitted CPU sets. The raw files are archived in `benchmarks/csf3/`.

products, 6.07 and 7.50 for LRT, and 4.65 and 5.25 for TRT. These are within-run comparisons with that specific single-threaded baseline.

The results support eight threads as a practical choice for this problem size. Moving from eight to sixteen reduced LRT and TRT times by about 6% and 10% in run A, but increased them by about 6% and 0.3% in run B. Thus sixteen threads showed no consistent tensor advantage. Variation between jobs and the changed CPU affinity prevent a precise claim about parallel efficiency or its hardware cause; these two runs do not establish a universal optimum or a statistical confidence interval.

At eight threads, geometry construction took 0.50–0.58 s and additional MKL preparation took 0.64–0.69 s. The process resident-memory increase across MKL preparation was about 1.05 GiB, while the complete comparison benchmark peaked at about 2.85 GiB. These measurements include allocations beyond the original CSR matrix and should not be interpreted as the memory required by a minimal MKL-only implementation. Charging the full MKL preparation cost against the measured savings over the cached-transpose baseline gives a break-even of approximately two LRT or TRT forward/adjoint pairs; this excludes common geometry construction and process/library startup.

Across both jobs the largest relative product discrepancy from SciPy was 5.6×10^{-16} or less, and the largest norm-scaled tensor adjoint discrepancy was 4.2×10^{-18} or less. This is evidence of agreement for these discrete test problems, not of continuum reconstruction accuracy. The measured speedups justify further evaluation of an optional cached MKL backend, with complete reconstruction time and memory as the next validation targets.

8 Reproduction and scope for further work

The guide's `README.md` gives a LaTeX build command and a script for regenerating the figures. The following commands run the existing examples and checks from the repository root in a suitable Python environment:

```
python -m unittest discover -s tests -v
cmake -S . -B build/cpp
cmake --build build/cpp
ctest --test-dir build/cpp --output-on-failure
python examples/trt_slices.py --json
python benchmarks/benchmark.py
# With an OpenMP build:
python benchmarks/atomic_adjoint.py --threads 1 2 4 8
```

Optional dependency tests may skip if their software is missing; a passing minimal installation is not evidence that CIL or ASTRA integration ran. The CI configuration provisions these integrations separately and builds the MATLAB MEX tests. Benchmarks report observations; they are not timing gates on shared CI machines.

Useful next steps include a documented acquisition-geometry converter for CIL, a separately tested TTRT adapter, additional external 3D comparisons, and memory-controlled tensor operators for problems too large to cache. Non-Abelian reconstruction needs physical measurement models and tested Jacobian/adjoint pairs. More advanced tensor inversions require their own specified data measure and sampling. ORT should receive a public definition before an implementation is advertised. These are extensions of a common geometric foundation, rather than capabilities implied by the present examples.

A Exact-arithmetic specification and correctness

This is a first step towards formal verification: a precise specification and a written mathematical proof of an exact-arithmetic model of `walk()`. It is **not yet a machine-checked proof**, a proof about the actual C++ floating-point execution, or a verification of the wrappers. The correspondence below was checked against `traverse.hpp` at revision `e1c4313`.

A *specification* states what a function must deliver, independently of its implementation. A *precondition* is an assumption on its inputs; a *postcondition* is a promised property of its output. A *loop invariant* is a statement that holds whenever execution reaches the top of the loop. Proving an invariant requires establishing it initially and showing that every iteration preserves it. Termination needs a separate argument.

A.1 Inputs and the meaning of an output record

Let $D \in \{2, 3\}$ be the dimension. We number coordinates from zero, to match the implementation. Throughout this appendix the coordinate index is an integer

$$i \in \{0, 1, \dots, D-1\} \subset \mathbb{Z}.$$

In 3D, $i = 0, 1, 2$ correspond respectively to the x, y, z directions. The grid inputs are:

- $n_i \in \mathbb{Z}_{>0}$: the *number of voxels* in direction i. Their indices are $0, 1, \dots, n_i - 1$. Thus $n_i - 1$ is the largest voxel index, not the number of voxels. There are $n_i + 1$ grid planes in that direction, numbered $0, 1, \dots, n_i$.
- $h_i \in \mathbb{R}_{>0}$: the physical width of each voxel in direction i.
- $o = (o_0, \dots, o_{D-1}) \in \mathbb{R}^D$: the lower corner of the grid.

The ray endpoints are $p, e \in \mathbb{R}^D$. Define its parametrisation by

$$r(t) = p + t(e - p), \qquad t \in [0, 1] \subset \mathbb{R}, \qquad L = \|e - p\|_2.$$

Here $r(0) = p$, $r(1) = e$, and L is the physical length of the whole input segment, including any portion outside the grid. For each coordinate define

$$q_i = \frac{p_i - o_i}{h_i}, \qquad v_i = \frac{e_i - p_i}{h_i}.$$

Thus $q_i + t v_i$ is the ray coordinate measured in voxel widths from o_i.

A voxel is labelled by an *integer vector*

$$c = (c_0, \dots, c_{D-1}) \in \mathbb{Z}^D, \qquad c_i \in \{0, 1, \dots, n_i - 1\} \quad \text{for every } i.$$

The vector c is an index, not a physical position. Its voxel has lower corner with coordinates $o_i + c_i h_i$. The grid and the voxel labelled by c are the sets

$$\Omega = \prod_{i=0}^{D-1} [o_i, o_i + n_i h_i), \qquad V_c = \prod_{i=0}^{D-1} [o_i + c_i h_i, o_i + (c_i + 1) h_i).$$

Here $\prod$ denotes the *Cartesian product of sets*. For example, $V_{(1,0)} = [1, 2) \times [0, 1)$ in a 2D unit grid at the origin. These half-open boxes partition Ω: a point on an internal face belongs to the higher-index voxel in that coordinate. The lower outer faces are included; the upper outer faces are excluded.

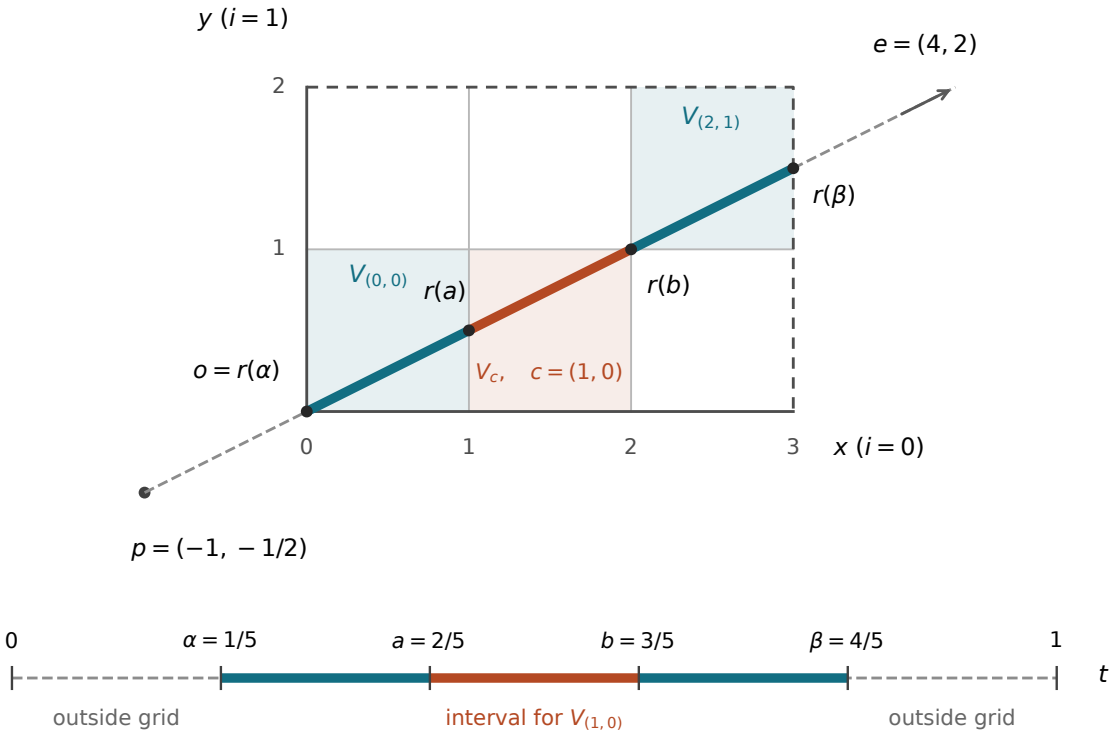


Figure 3: Notation for a two-dimensional unit grid: $D = 2$, $n = (3, 2)$, $h = (1, 1)$ and $o = (0, 0)$. The ray runs from $p = (-1, -1/2)$ to $e = (4, 2)$. The orange portion passes through voxel $c = (1, 0)$, with entry parameter $a = 2/5$ and exit parameter $b = 3/5$. The same portion is marked on the parameter line below. The grid entry and exit are $\alpha = 1/5$ and $\beta = 4/5$. Solid lower/left and dashed upper/right outer edges indicate the half-open grid boundary.

The *output* is an ordered list of entries, one per positive-length voxel intersection. We call one entry an *output record*: "record" is the programming term for a group of related data. In our mathematical model it is the tuple

$$(c, a, b, \ell), \qquad a, b \in \mathbb{R}, \quad 0 \le a < b \le 1, \quad \ell = (b - a)L,$$

where c identifies the voxel, a, b are its ray entry and exit parameters, and ℓ is the physical length travelled in it. Thus a record is one item in the output, not a separate output. The C++ function reports each intersection by calling the supplied function `emit`; it does not itself construct a list of these tuples. It passes a linear voxel index, the length ℓ, and the entry and exit parameters a, b to `emit`. The index is $c_0 + n_0(c_1 + n_1 c_2)$ in 3D, or $c_0 + n_0 c_1$ in 2D.

For an entry (c, a, b, ℓ) we require $r(t) \in V_c$ for every $a < t < b$. **We do not require both endpoints $r(a)$ and $r(b)$ to belong to V_c.** The parameters mark the ends of the portion traversed in the voxel. If an endpoint lies on a voxel boundary, its set membership is determined by the half-open convention. In Figure 3, $r(b) = (2, 1)$ does not belong to $V_{(1,0)} = [1, 2) \times [0, 1)$, although b is correctly recorded as its exit parameter. It belongs to $V_{(2,1)}$. For a ray travelling in the negative x direction across $x = k$, the next interval lies in column $k - 1$, although the crossing point itself lies in column k. These isolated points have zero length and do not affect the reported intersection lengths.

The required postconditions are:

1. If $L = 0$, the output is empty. Otherwise, emit exactly one record for each voxel whose ray intersection has positive length, and no others.
2. Each record has $0 \le a < b \le 1$, a valid cell index, and $r(t) \in V_c$ for every $a < t < b$. Its interval is the full positive-length intersection with that voxel, up to endpoints.
3. Records occur in increasing parameter order. Consecutive intervals meet at an endpoint; their interiors are disjoint. Together they cover the part of the ray in Ω, apart from finitely many endpoints.
4. Each length is exact. In particular, writing $|S|$ for the one-dimensional measure of a parameter set,
$$\ell_c = L\,|\{t \in [0,1] : r(t) \in V_c\}|, \qquad \sum_c \ell_c = L\,|\{t \in [0,1] : r(t) \in \Omega\}|.$$
5. The algorithm terminates, with at most $1 + \sum_i (n_i - 1)$ emitted records when the output is nonempty.

A.2 The exact algorithm corresponding to `walk()`

Replace floating-point arithmetic by exact real arithmetic and fixed-width indices by mathematical integers. Interpret `fma(first,v,q)` as $q + \operatorname{first} v$, and the nested `hypot` calls as L. These replacements define the model; their correctness for machine arithmetic is *not* assumed to have been proved. The C++ core uses three coordinates; the 2D case is represented by adding a constant third coordinate inside a layer one voxel thick. The supplied function `emit` is called a *callback*: the traversal calls it once for each intersection. Here its only action is to append a record to the output list. Any later proof using an actual callback must specify its termination and effects on inputs.

Clipping means restricting the ray to the part inside the grid. Return immediately if $L = 0$. A direction with $v_i = 0$ is called stationary: the ray does not move in that coordinate. In such a direction, return empty if $q_i \notin [0, n_i)$; otherwise it imposes no parameter restriction. For each moving direction, the condition $0 \le q_i + t v_i \le n_i$ restricts t to a closed interval with endpoints $-q_i/v_i$ and $(n_i - q_i)/v_i$. Intersect these intervals with $[0,1]$, calling the resulting lower and upper bounds α and β. Return empty if $\alpha \ge \beta$.

Set $t = \alpha$. Write $x_i = q_i + \alpha v_i$ for the entry coordinate in voxel units, and initialise
$$c_i = \begin{cases} \lfloor x_i \rfloor, & v_i \ge 0, \\ \lceil x_i \rceil - 1, & v_i < 0, \end{cases} \qquad \sigma_i = \operatorname{sgn}(v_i).$$

The next crossing parameter is
$$\tau_i = \begin{cases} (c_i + 1 - q_i)/v_i, & v_i > 0, \\ (c_i - q_i)/v_i, & v_i < 0, \\ +\infty, & v_i = 0. \end{cases}$$

For a stationary direction there is no next face crossing, so we use $+\infty$ to express this. It is larger than every finite crossing parameter and cannot be selected instead of the finite endpoint β in a minimum. A special value used in this way is called a *sentinel* in programming. Here $+\infty$ participates only in comparisons; no arithmetic is done with it.

While $t < \beta$, set $s = \min(\beta, \tau_0, \ldots, \tau_{D-1})$, emit $(c, t, s, (s - t)L)$, and set $t = s$. If $s = \beta$, finish. Otherwise, for *every* axis with $\tau_i = s$, replace c_i by $c_i + \sigma_i$ and recompute τ_i using the formula above.

The C++ implementation in `traverse.hpp` also uses two `min`/`max` operations to restrict values to an allowed range. This is called *clamping*: a value below the lower bound is replaced by that bound, a value above the upper bound is replaced by the upper bound, and a value already in range is unchanged. Here the computed entry coordinate is restricted to $[0, n_i]$, and the initial voxel index to $[0, n_i - 1]$. In exact arithmetic these operations leave both values unchanged. Also, the condition `stop > t` holds in every entered loop iteration, and the early return for an out-of-grid index cannot occur before the final endpoint. The proof below establishes these facts; they are not extra preconditions.

A.3 Correctness proof

Theorem. Under the preconditions above, the exact algorithm satisfies all five postconditions.

1. Clipping. If $v_i = 0$, membership in that coordinate slab is constant and is exactly the test $0 \le q_i < n_i$. If $v_i \neq 0$, the inverse image of the closed slab $[0, n_i]$ is the interval computed above. Replacing the actual half-open slab by the closed slab changes membership only at its two endpoint parameters. Hence, after the stationary-axis tests, the parameter set inside the grid differs from the intersection of these closed intervals with $[0, 1]$ by at most finitely many points. It has positive length exactly when $\alpha < \beta$. Every parameter strictly between these bounds lies in Ω. The empty returns, including point tangencies, are therefore correct. The separately handled $L = 0$ case has zero physical length.

2. Initialisation. Assume now that $L > 0$ and $\alpha < \beta$. For $v_i > 0$, clipping gives $0 \le x_i < n_i$: equality at the upper end would leave no interior interval. Thus $\lfloor x_i \rfloor$ is a valid cell, occupied immediately after α. For $v_i < 0$, similarly $0 < x_i \le n_i$, and $\lceil x_i \rceil - 1$ is the valid cell occupied immediately after α. For $v_i = 0$, the stationary-axis test gives $0 \le x_i < n_i$, and $\lfloor x_i \rfloor$ gives the prescribed voxel assignment. Both range-restriction operations in the C++ implementation consequently leave these initial values unchanged. For a moving axis, τ_i is strictly larger than α and is its first cell-face crossing after α.

3. Invariant. At the top of every entered iteration:

(i) $\alpha \le t < \beta$, and the records already emitted correctly cover the parameter interval from α to t, up to endpoints.

(ii) c is a valid cell occupied immediately after t.

(iii) Each moving axis has $\tau_i > t$, its first forward-in-parameter face crossing of this cell; stationary axes have $\tau_i = +\infty$.

Here "immediately after" means that some ε with $0 < \varepsilon \le \beta - t$ exists such that $r(u) \in V_c$ for every $u \in (t, t + \varepsilon)$. Initialisation proves the invariant before the first iteration, with an empty output list.

4. Preservation and the emitted interval. By (iii), $s > t$. No coordinate crosses a cell face for $t < u < s$, so the entire open interval lies in V_c. Its physical length is exactly $(s - t)L$. Appending it preserves ordered coverage and creates no overlap of interiors. This is the maximal remaining interval in that cell: at s either the clipped ray ends or at least one coordinate leaves the cell.

If $s < \beta$, at least one moving axis has $\tau_i = s$. Each such crossing is of an *internal* plane: an outward crossing of the outer box would end the clipped interval at or before s, contradicting $s < \beta$. Updating every tied axis therefore gives valid indices and the cell occupied immediately after s. A tied axis's next crossing increases by $1/|v_i| > 0$; untied axes already have $\tau_i > s$. Thus (i)–(iii) hold for the next iteration. Since all $\tau_i \ge s$ before updating, the C++ test $\tau_i \le s$ selects exactly the tied axes. No record is emitted between the individual axis updates.

5. Termination. The nonnegative integer

$$R(c) = \sum_{i:v_i>0} (n_i - 1 - c_i) + \sum_{i:v_i<0} c_i$$

counts the internal planes still ahead, whether or not the finite ray will reach them. Every nonfinal iteration decreases R by the number of tied axes, hence by at least one. This is a *loop variant*: a quantity in the nonnegative integers that strictly decreases on each continuation. There cannot be infinitely many continuations. Since initially $R \le \sum_i (n_i - 1)$, there are at most that many nonfinal iterations and one final emission. The last emission ends at β.

6. Completeness and uniqueness. The invariant and termination give contiguous coverage from α to β, up to endpoints. Each moving cell coordinate only changes in its fixed direction, and some coordinate changes between consecutive records; therefore no cell can be visited twice. Every positive-length voxel intersection contains an open parameter interval and so must be represented in this coverage. Consequently the emitted records are exactly the required intersections. Summing their lengths gives $L(\beta - \alpha)$, which equals the length in the grid by the clipping argument. Valid cell coordinates also imply a linear index between zero and $\prod_i n_i - 1$ in mathematical integer arithmetic. This proves all the postconditions. □

A.4 A small example with a simultaneous crossing

Return to the 3×2 unit grid in Figure 3, with $p = (-1, -\frac{1}{2})$, $e = (4, 2)$. Then

$$r(t) = (-1 + 5t, -\tfrac{1}{2} + \tfrac{5}{2}t), \quad \alpha = \tfrac{1}{5}, \quad \beta = \tfrac{4}{5}, \quad L = \tfrac{5}{2}\sqrt{5}.$$

The exact output is

Cell c	Linear index	Entry a	Exit b	Length
$(0, 0)$	0	1/5	2/5	$\sqrt{5}/2$
$(1, 0)$	1	2/5	3/5	$\sqrt{5}/2$
$(2, 1)$	5	3/5	4/5	$\sqrt{5}/2$

At $t = 3/5$ the ray crosses $x = 2$ and $y = 1$ simultaneously, so both cell coordinates advance. Cells $(2, 0)$ and $(1, 1)$ receive no positive-length interval. The initial variant is $R = 2 + 1 = 3$; the first crossing reduces it to 2 and the simultaneous crossing reduces it to 0. The next record ends at the grid boundary. Reversing the ray reverses the cell list and maps every interval (a, b) to $(1 - b, 1 - a)$, preserving its length.

A.5 What remains to be formalised

The first Lean milestone would be the one-coordinate initialisation and next-crossing lemmas, followed by the invariant and termination argument. For rational inputs, all crossing parameters are rational; the possibly irrational length L can be treated separately. This would verify a formal model, which would still need to be connected to the C++ implementation.

The present proof excludes floating-point rounding, overflow/underflow, fixed-width indices and casts, validators, library-function accuracy, callbacks, wrappers and parallel accumulation. Tests comparing a model with C++ would add useful evidence, but would not establish their equivalence for all inputs.

References

[1] ASTRA Toolbox contributors. ASTRA: 2d projectors, 2026. URL https://astra-toolbox.com/docs/proj2d.html. Documentation, accessed 27 October 2026.

[2] Ander Biguri, Manjit Dosanjh, Steven Hancock, and Manuchehr Soleimani. TIGRE: a MATLAB-GPU toolbox for CBCT image reconstruction. *Biomedical Physics & Engineering Express*, 2(5):055010, 2016. doi: 10.1088/2057-1976/2/5/055010.

[3] CIL developers. Core Imaging Library documentation, 2026. URL https://tomographicimaging.github.io/CIL/. Installation instructions; accessed 27 October 2026.

[4] CIL developers. CIL TIGRE ProjectionOperator documentation and source, 2026. URL https://tomographicimaging.github.io/CIL/v24.1.0/_modules/cil/plugins/tigre/ProjectionOperator/. Version 24.1.0; accessed 27 October 2026.

[5] Naeem M. Desai and William R. B. Lionheart. An explicit reconstruction algorithm for the transverse ray transform of a second rank tensor field from three axis data. *Inverse Problems*, 32 (11):115009, 2016. doi: 10.1088/0266-5611/32/11/115009. URL https://arxiv.org/abs/1511.05305.

[6] Naeem M. Desai, William R. B. Lionheart, Morten Sales, Markus Strobl, and Søren Schmidt. Polarimetric neutron tomography of magnetic fields: uniqueness of solution and reconstruction. *Inverse Problems*, 36(4):045001, 2020. doi: 10.1088/1361-6420/ab44e0. URL https://arxiv.org/abs/1903.10884.

[7] Tommy Elfving and Per Christian Hansen. Unmatched projector/backprojector pairs: Perturbation and convergence analysis. *SIAM Journal on Scientific Computing*, 40(1):A573–A591, 2018. doi: 10.1137/17M1133828.

[8] Per Christian Hansen, Ken Hayami, and Keiichi Morikuni. GMRES methods for tomographic reconstruction with an unmatched back projector. *Journal of Computational and Applied Mathematics*, 413:114352, 2022. doi: 10.1016/j.cam.2022.114352.

[9] F. Jacobs, E. Sundermann, B. De Sutter, M. Christiaens, and I. Lemahieu. A fast algorithm to calculate the exact radiological path through a pixel or voxel space. *Journal of Computing and Information Technology*, 6(1):89–94, 1998. URL https://biblio.ugent.be/publication/281913.

[10] J. S. Jørgensen, E. Ametova, G. Burca, G. Fardell, E. Papoutsellis, E. Pasca, K. Thielemans, M. Turner, R. Warr, W. R. B. Lionheart, and P. J. Withers. Core imaging library—part I: a versatile Python framework for tomographic imaging. *Philosophical Transactions of the Royal Society A*, 379(2204):20200192, 2021. doi: 10.1098/rsta.2020.0192.

[11] William R. B. Lionheart. LRTinMatlab: research code containing David Szotten's ray implementation. GitHub repository, 2026. URL https://github.com/billlion/LRTinMatlab/tree/4e61af83e3d9cfe58305cb1b0fe95532ca1aea94. Inspected revision 4e61af83e3d9cfe58305cb1b0fe95532ca1aea94; accessed 27 October 2026.

[12] William R. B. Lionheart and Vladimir A. Sharafutdinov. Reconstruction algorithm for the linearized polarization tomography problem with incomplete data. In *Imaging Microstructures*, volume 494 of *Contemporary Mathematics*, pages 137–159. American Mathematical Society, 2009. doi: 10.1090/conm/494/09648. URL https://eprints.maths.manchester.ac.uk/1247/.

[13] William R. B. Lionheart and Philip J. Withers. Diffraction tomography of strain. *Inverse Problems*, 31(4):045005, 2015. doi: 10.1088/0266-5611/31/4/045005.

[14] Morten Sales, Markus Strobl, Takenao Shinohara, Anton Tremsin, Luise Theil Kuhn, William R. B. Lionheart, Naeem M. Desai, Anders Bjorholm Dahl, and Søren Schmidt. Three dimensional polarimetric neutron tomography of magnetic fields. *Scientific Reports*, 8:2214, 2018. doi: 10.1038/s41598-018-20461-7.

[15] Vladimir A. Sharafutdinov. *Integral Geometry of Tensor Fields*. Inverse and Ill-Posed Problems. VSP, Utrecht, 1994.

[16] Robert L. Siddon. Fast calculation of the exact radiological path for a three-dimensional CT array. *Medical Physics*, 12(2):252–255, 1985. doi: 10.1118/1.595715.

[17] David Szotten. *Limited Data Problems in X-ray and Polarized Light Tomography*. PhD thesis, The University of Manchester, 2011. URL https://research.manchester.ac.uk/en/studentTheses/limited-data-problems-in-x-ray-and-polarized-light-tomography/. Year as cited in the TRT and neutron papers; the university catalogue records the award on 3 December 2010.

[18] David Szotten, William R. B. Lionheart, and Rachel A. Tomlinson. Tomographic reconstruction of stress from photoelastic measurements using elastic regularization. MIMS EPrint 2006.5, Manchester Institute for Mathematical Sciences, 2006. URL https://eprints.maths.manchester.ac.uk/143/1/matrix_photoelast.pdf. Preprint.

[19] W. van Aarle, W. J. Palenstijn, J. Cant, E. Janssens, F. Bleichrodt, A. Dabravolski, J. De Beenhouwer, K. J. Batenburg, and J. Sijbers. Fast and flexible X-ray tomography using the ASTRA toolbox. *Optics Express*, 24(22):25129–25147, 2016. doi: 10.1364/OE.24.025129.